

Autonomous 4WD Vehicle

Zewail City of Science and Technology

Course Code: CIE 408

Course Name: Embedded Systems

Instructor: Dr. Hazem Fahmy

Team Members:

- Nada Mohamed 202200635
- Ashar salama 202200199
- Reem Tamer 202300845
- Malak Osama 202200444
- Alya Marwan 202200375

Platform: PIC16F877A + Raspberry Pi 3

Communication: UART 9600 baud · Firebase RTDB

AI Models: Anomaly Detection · Avoidance Classifier · Voice NLP

GitHub Repository: Autonomous-Robot

Table of Contents

1	Executive Summary	3
2	System Architecture	3
2.1	Dual-Processor Design	3
2.2	UART Protocol	4
2.3	Hardware Schematic	5
3	PIC16F877A Firmware	5
3.1	PWM Motor Control	5
3.2	Ultrasonic Sensor Driver — Round-Robin Triggering	6
3.3	Stack Depth Management	6
4	Three-Layer Safety System	6
4.1	Failsafe 1 — Automatic Emergency Braking (AEB)	7
4.2	Failsafe 2 — Dead Man Switch	7
4.3	Failsafe 3 — Traction Control	7
5	PID Controller	7
5.1	Purpose and Equation	7
5.2	Straight-Line Correction	8
6	AI / Machine Learning System	8
6.1	Model 1 — Anomaly Detection (Isolation Forest)	8
6.1.1	Purpose	8
6.1.2	Features	8
6.1.3	Training Implementation	8
6.2	Model 2 — Obstacle Avoidance (Random Forest Classifier)	9
6.2.1	Purpose	9
6.2.2	Input / Output Configuration	9
6.2.3	Training Pipeline	9
6.2.4	ONNX Inference Implementation	10
6.3	Model 3 — Voice Command Intent Classifier (NLP Layer)	10
6.3.1	Purpose	10
6.3.2	Intent Extraction Framework	10
6.3.3	Deployment Architecture	10
6.3.4	Autonomous Control Fallback Hierarchy	11
7	Raspberry Pi Software	11
7.1	Thread Architecture	11
7.1.1	Atomic State Preservation Fix	12
7.2	Dead-Reckoning Odometry	12
7.3	Elaborative Analysis of the Parallel Auto-Parking Subsystem	12
7.3.1	Kinematic Constraints of Differential-Drive Parallel Parking	12
7.3.2	The 5-Phase Parallel Parking Finite State Machine Architecture	12
7.4	FSM	13
7.5	FSM	13
8	Firestore & Dashboard	15
8.1	Database Structure	15
8.2	Voice Control Flow	15
9	PCB Design & Hardware Debugging	16
9.1	Assembled Prototype	16
9.2	PCB Design Views	16
9.3	Systematic Bring-Up Checklist	18

10 Key Bugs and Fixes	19
11 Team Workload Division	20
12 Conclusion and Future Work	20
12.1 Achievements	20
12.2 Future Work	20

1. Executive Summary

This report documents the design, implementation, and testing of an advanced autonomous 4-wheel-drive vehicle using a dual-processor architecture. A **PIC16F877A** microcontroller serves as the real-time reflexes layer, while a **Raspberry Pi 3** provides high-level intelligence, cloud connectivity, and AI inference.

The system implements three independent hardware safety failsafes, a live Firebase telemetry dashboard with voice control, an automatic parallel-parking state machine, and three machine learning models for anomaly detection, obstacle avoidance, and vocal intent classification.

Key Achievements

- Three simultaneous hardware safety failsafes — AEB, Dead Man Switch, Traction Control
- Live Firebase dashboard with voice control via Google Speech API and native intent mapping
- Automatic parallel-parking finite state machine with active sensor-based centering adjustments
- AI Anomaly Detection (Isolation Forest) for motor stall, wheel slip, and sensor fault isolation
- AI Obstacle Avoidance (Random Forest) with optimized ONNX execution on the Raspberry Pi 3
- AI Voice Intent Classifier utilizing multi-class natural language keyword matching tokens
- PID controller for wheel speed regulation and straight-line trajectory error correction
- Round-Robin ultrasonic triggering sequence eliminating acoustic cross-talk interference
- Custom PCB layout design, component assembly, and systematic hardware level-shifter debugging

2. System Architecture

2.1 Dual-Processor Design

Linux running on a Raspberry Pi is not a Real-Time Operating System (RTOS) and cannot guarantee the microsecond-accurate signal capturing required by high-frequency hardware drivers. To resolve this limitation, the computational tasks are split cleanly across an intelligent multi-threaded high-level processor and an independent real-time microcontroller.

The **Raspberry Pi 3** runs a Python-driven brain system that manages asynchronous thread tasks, loads trained machine learning matrix files via an optimized ONNX runtime environment, computes localization mapping values, and establishes secure cloud sockets with a remote Firebase web platform. The **PIC16F877A** functions as a dedicated hardware reflexes layer. It runs bare-metal C routines designed to handle raw hardware pin capture, process high-frequency input capture states, toggle motor driver logic pins, and coordinate safety overrides with zero operating system overhead.

Table 1: Processor responsibility split

PIC16F877A — Reflexes	Raspberry Pi 3 — Brain
HC-SR04 echo timing ($1\ \mu\text{s}$ via Timer1)	Python control loop, AI inference, parking FSM
10 kHz PWM motor speed (CCP1/CCP2)	Firestore RTDB telemetry push every 300 ms
Encoder pulse counting (INT0 + IOC)	Google Speech voice parsing and intent token classification
UART command reception + ring buffer	ONNX Runtime obstacle avoidance inference
Three hardware safety failsafes	Dead-reckoning odometry kinematics from wheel RPM
9-byte telemetry frame every 96 ms	Auto-parking multi-phase parallel state machine

2.2 UART Protocol

Data exchange between the processors occurs over a hardwired asynchronous serial interface running at a standard configuration of **9,600 baud**.

Pi → PIC: 2-byte command packet [CMD 0x00–0x05] [SPEED 0–100]

Every downstream instruction packed by the Raspberry Pi is bundled into a fixed 2-byte structure to minimize serial overhead.

PIC → Pi: 9-byte telemetry frame (every 96 ms)

[0xAA][Front cm][Back cm][Right cm][LRPM_H][LRPM_L][RRPM_H][RRPM_L][0x55]

The upstream communication uses static boundary tokens (0xAA header and 0x55 footer) to enable a sliding-window parser loop on the Raspberry Pi. This loop handles data stream framing reliably, even when faced with buffer delays or partial packet delivery caused by operating system scheduling latency.

Table 2: Command byte definitions

CMD	Name	Action
0x00	STOP	Halt all motor channels immediately
0x01	FORWARD	Drive IN pins forward at SPEED% duty cycle
0x02	REVERSE	Drive IN pins backward at SPEED% duty cycle
0x03	LEFT	Tank-turn left at SPEED%
0x04	RIGHT	Tank-turn right at SPEED%
0x05	BEEP	PIC plays double-beep stall alert via hardware buzzer

2.3 Hardware Schematic

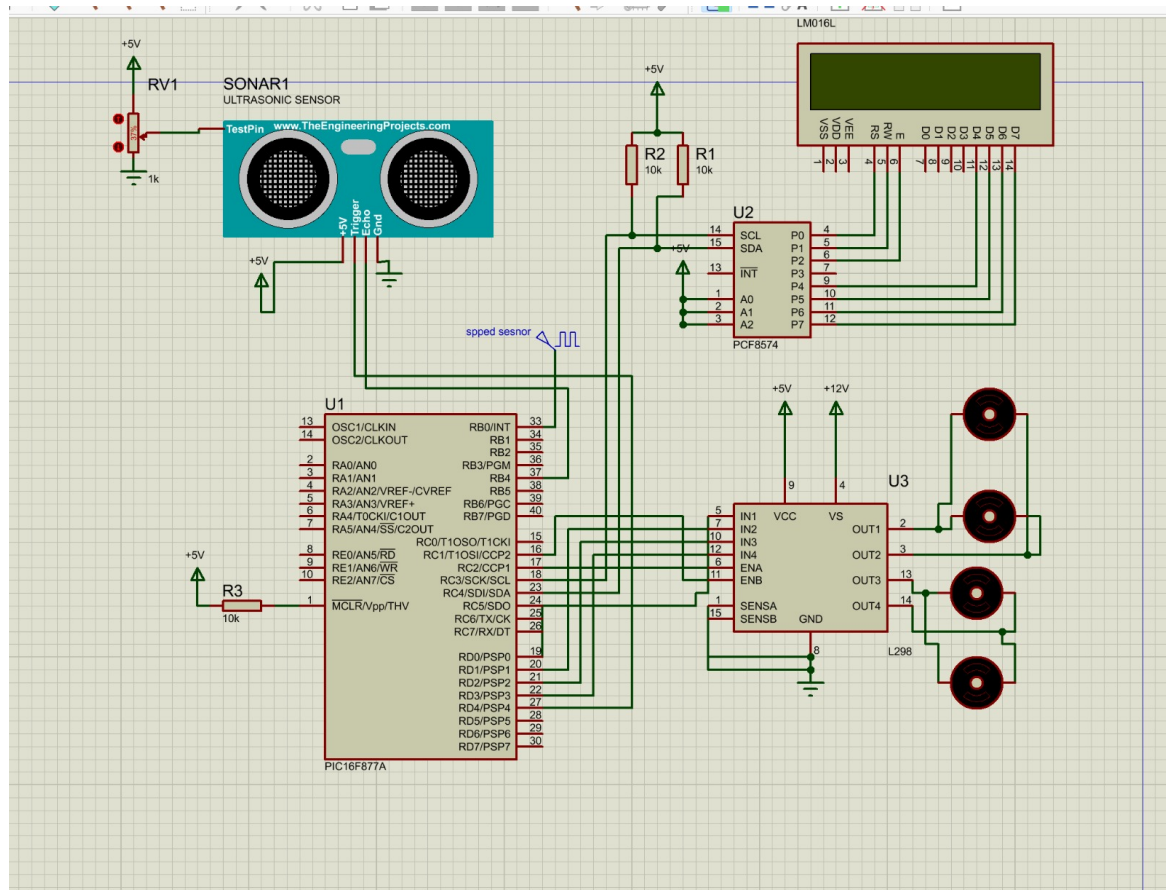


Figure 1: Proteus simulation schematic — PIC16F877A, L298N, HC-SR04, BSS138

3. PIC16F877A Firmware

3.1 PWM Motor Control

The system's original configuration utilized a 1 kHz PWM driving frequency. Under testing, this introduced severe inductive coils choke, where the motor winding inductive time constant ($\tau = L/R \approx 1 \text{ ms}$) matched the PWM period exactly. During a 50% duty cycle command cycle, the armature current was interrupted too quickly, only reaching:

$$I = \left(1 - e^{-0.5 \text{ ms}/1 \text{ ms}}\right) \times I_{\text{max}} = 39\% I_{\text{max}}$$

This caused the motors to output a loud audible whine while generating insufficient physical torque to overcome the stationary inertia of the 4WD chassis. To resolve this limitation, the driving frequency was raised to 10 kHz ($T = 100 \mu\text{s} \ll \tau$), smoothing the current waveform into an effective DC state across all speed ranges and providing consistent mechanical torque.

Two PWM Bugs Fixed (PWM_config.h + PWM.c)

Bug 1 — PR2 overflow: Setting $\text{FREQ}=5000$, $\text{PRESCALE}=1$ on the new 20 MHz crystal clock yielded a calculated period step parameter of:

$$\text{PR2} = \frac{20\,000\,000}{4 \times 1 \times 5000} - 1 = 999$$

This value exceeded the 8-bit limit of the hardware PR2 register (max 255), causing it to silently wrap around to 231 and mangle the target driving output frequency.

Fix: The timer variables were scaled to `FREQ=10000`, `PRESCALE=4`, shifting the register parameters into safe boundaries:

$$\text{PR2} = \frac{20\,000\,000}{4 \times 4 \times 10000} - 1 = 124$$

Bug 2 — Timer2 never started: The initialization routine configured the mode registers but omitted starting the base timer. Because the `T2CON.TMR2ON` bit remained cleared, the CCP modules had no reference clock, keeping the physical motor enable lines pinned permanently LOW (0 V).

Fix: Added an explicit macro statement `SET_BIT(T2CON, TMR2ON)` at the tail of the initialization block.

3.2 Ultrasonic Sensor Driver — Round-Robin Triggering

Three HC-SR04 sensors (front, back, right) are interfaced with the microcontroller, sharing the same Input Change Notification (IOC) interrupt cluster on pins RB5, RB6, and RB7. High-precision 1 μ s echo capture is handled via hardware Timer1 tracking loops.

Firing all three ultrasonic transducers at the same instant caused severe acoustic cross-interference, where the ultrasonic sound waves from the right sensor bounced off adjacent walls and tripped the echo receiver of the front sensor. The firmware solves this by enforcing a deterministic *Round-Robin sequential scheduling timeline*:

`Front Sensor (0ms) → Back Sensor (96ms) → Right Sensor (192ms) → repeat`

This creates a full refresh period of 288 ms per sensor, ensuring that each ultrasonic wave completely dissipates in the environment before the next sensor is allowed to fire.

PORTB Race Condition Fix: Reading the PORTB register twice inside the interrupt service routine to check different pin states can cause the microcontroller to clear the internal mismatch latch prematurely, dropping adjacent encoder pulses. The firmware fixes this by taking an isolated snapshot of PORTB at the entry point of the ISR, passing that cached state down to the execution handlers: `Ultrasonic_IOC_ISR(portb_val)`.

3.3 Stack Depth Management

The hardware stack inside the PIC16F877A is constrained to an absolute depth limit of 8 levels. Because deep nesting of nested function calls during nested interrupts causes standard stack overflow and forces unpredictable chip resets, the entire architecture was refactored to minimize call branches.

Table 3: Stack depth reduction — PIC limit is 8 levels

Change	Before	After
UART ISR: inlined vs <code>UART_ISR()</code> →callback	depth 3 from ISR	depth 1
Safety_Check: <code>H_Bridge_Stop()</code> vs <code>CutMotors()</code>	depth 2 from main	depth 1
Total maximum depth	9 (overflow)	6 (safe)

4. Three-Layer Safety System

All three failsafes operate simultaneously in the bare-metal microcontroller layer, serving as independent hardware safety layers that cannot be disabled by high-level software overrides.

4.1 Failsafe 1 — Automatic Emergency Braking (AEB)

The low-level firmware monitors proximity inputs in real time to prevent collisions caused by communication dropouts or latency spikes in the high-level operating system.

- **Trigger Matrix:** An active emergency state is declared if the front distance falls within the range: $3\text{ cm} \leq \text{Front} < 15\text{ cm}$ while a forward command vector (`CMD_FORWARD`) is being executed.
- **Immediate Response:** The microcontroller intercepts the motor control registers, executing an atomic `H_Bridge_Stop()` routine and pinning the buzzer output HIGH (`Buzzer_On()`) to alert operators.
- **Automatic Resume Integration:** When an obstacle triggers an AEB stop, the firmware clears the incoming queue but preserves the target status state memory. Once the path is clear, the car automatically resumes forward movement without requiring user manual resetting, eliminating the stutter-lock bug of previous versions.

4.2 Failsafe 2 — Dead Man Switch

The Raspberry Pi maintains a continuous 5 Hz keepalive heartbeat stream. Every valid command packet processed by the serial interrupt loop automatically resets a hardware countdown variable: `heartbeat_ticks = 0`.

The safety tracking logic evaluates this counter against a hard limit:

$$\text{Timeout} = 50 \times 16\text{ ms} = 800\text{ ms}$$

This provides a safe 300 ms padding layer over the nominal 500 ms heartbeat arrival rate. If a serial line breaks, the level shifter disconnects, or the Pi's operating system freezes, the counter hits its limit, causing the PIC to instantly cut power to the H-bridge pins and flash the alarm buzzer at 320 ms cycles.

Heartbeat Multi-Byte Packet Bug Fixed

The original version sent a single raw byte (`0xFF`) as its heartbeat token. If this byte arrived when the PIC's state machine was at `rx_stage == 1` (waiting for a speed value), the `0xFF` token was mistakenly treated as a speed parameter, commanding 100% motor speed unexpectedly.

Fix: The heartbeat thread was updated to always transmit a complete, valid 2-byte structure: `[CMD_STOP, 0x00]`.

4.3 Failsafe 3 — Traction Control

Infrared optocoupler speed sensors mounted on the left and right wheel assemblies feed raw digital pulses back into the microcontroller via the `INT0 (RB0)` pin and an `I0C (RB4)` channel. The traction control library tracks these pulse registers across a strict 16 ms time window managed by `Timer0`.

If the tracking logic registers an unexpected single-wheel RPM spike (indicating wheel slip) or a severe left-to-right wheel speed imbalance while driving straight, it automatically cuts power to prevent a spin-out or structural drift.

5. PID Controller

5.1 Purpose and Equation

A closed-loop discrete PID controller implemented across `PID.c` and `TC_Module.c` dynamically regulates individual motor velocities under changing payload conditions, ensuring the vehicle maintains a true straight line.

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{e(t) - e(t - \Delta t)}{\Delta t}$$

Table 4: PID parameter roles

Parameter	Symbol	Effect
Proportional gain	K_p	Immediate correction proportional to current speed error
Integral gain	K_i	Eliminates steady-state offset over time
Derivative gain	K_d	Predicts error trends to damp oscillations and overshoot
Update interval	16 ms	Synced to the Timer0 RPM tracking window
Output clamp	0–100%	Caps actuator values to prevent integral windup

5.2 Straight-Line Correction

When the vehicle is commanded to drive straight, any asymmetric track friction or differences in motor internal resistance will cause the chassis to drift. The controller monitors this by calculating a differential error metric:

$$e_{\text{diff}}(t) = \text{RPM}_L(t) - \text{RPM}_R(t)$$

This error value is fed back into the motor control loops. The duty cycle of the faster wheel is decreased while the slower wheel is accelerated, ensuring the vehicle maintains a straight path without manual steering inputs.

6. AI / Machine Learning System

Three independent machine learning models are deployed on the Raspberry Pi 3 platform using scikit-learn, with physical inference handled via optimized ONNX runtimes.

6.1 Model 1 — Anomaly Detection (Isolation Forest)

6.1.1 Purpose

This model monitors operating safety by identifying environmental and mechanical behaviors that deviate from normal driving profiles. It isolates hard-to-code faults such as dead motor stalls, floating pin noise, wheel slip, and the "kidnapped robot" scenario (where wheels spin at high velocity but ultrasonic distances remain static).

6.1.2 Features

The input data forms a 5-dimensional feature matrix captured at 10 Hz by a background logging thread:

$$X_{\text{anomaly}} = [\text{Front}, \text{Back}, \text{Right}, \text{Left_RPM}, \text{Right_RPM}]$$

6.1.3 Training Implementation

```
from sklearn.ensemble import IsolationForest
import pandas as pd

# Load structural logs
df = pd.read_csv('ai_training_data.csv')
X_normal = df[df['Source'] == 'HUMAN'][['Front', 'Back', 'Right', 'LeftRPM', 'RightRPM']]

# Configure isolation parameters
iso_forest = IsolationForest(contamination=0.04, random_state=42)
```

```
iso_forest.fit(X_normal)

# Export trained estimator parameters to ONNX matrix format
```

The model maps the dataset into isolation trees. During live runtime, any input vector that yields an anomalous score lower than a configured threshold triggers an immediate vehicle shutdown.

6.2 Model 2 — Obstacle Avoidance (Random Forest Classifier)

6.2.1 Purpose

This model functions as a supervised classification brain that maps proximity inputs to optimal motion vectors. It replaces arbitrary, hardcoded safety cut-offs with smooth, learned decision boundaries derived from real human driving data.

6.2.2 Input / Output Configuration

Input telemetry fields are mapped into a standardized range ($0.0 \leq x \leq 1.0$) by dividing raw data against an upper clip boundary parameter (200 cm).

Table 5: Model 2 features and output classes

Type	Name	Description
Input	front_norm	Front distance / 200 (0–1 range)
Input	back_norm	Back distance / 200 (0–1 range)
Input	right_norm	Right distance / 200 (0–1 range)
Output 0	CMD_STOP	All paths blocked; bring car to a safe halt
Output 1	CMD_FORWARD	Path clear ahead; proceed forward
Output 2	CMD_REVERSE	Blocked ahead; initiate backward clearance
Output 3	CMD_LEFT	Higher obstacle clearance left; pivot left
Output 4	CMD_RIGHT	Higher obstacle clearance right; pivot right

6.2.3 Training Pipeline

The model is trained on human driving logs, filtering out automated rule corrections to ensure the neural network only learns from clean human choices.

```
from sklearn.ensemble import RandomForestClassifier
from skl2onnx import to_onnx
from skl2onnx.common.data_types import FloatTensorType

# Read and isolate clean human data records
df = pd.read_csv('ai_training_data.csv')
df = df[df['Source'] == 'HUMAN']

X = df[['Front', 'Back', 'Right']].clip(upper=200) / 200.0
y = df['Command'].astype(int)

# Train Random Forest Ensemble
clf = RandomForestClassifier(n_estimators=100, max_depth=8, random_state=42)
clf.fit(X, y)

# Convert to a universal deployment ONNX model
initial_type = [('float_input', FloatTensorType([None, 3]))]
onnx_model = to_onnx(clf, X[:1].astype(np.float32), initial_types=initial_type)

with open("car_avoidance.onnx", "wb") as f:
    f.write(onnx_model.SerializeToString())
```

6.2.4 ONNX Inference Implementation

The Raspberry Pi runs real-time inference on the trained ONNX model file using CPU execution loops at 5 Hz:

```
import onnxruntime as ort
import numpy as np

# Load model weights onto the Pi
session = ort.InferenceSession('car_avoidance.onnx', providers=[
    'CPUExecutionProvider'])
input_name = session.get_inputs()[0].name

# Package scaled sensor data
obs = np.array([[front/200.0, back/200.0, right/200.0]], dtype=np.float32)
raw_outputs = session.run(None, {input_name: obs})[0][0]

# Extract the target motion vector index
predicted_cmd = int(np.argmax(raw_outputs))
send_command(ser_global, predicted_cmd, AVOID_SPEED)
```

6.3 Model 3 — Voice Command Intent Classifier (NLP Layer)

6.3.1 Purpose

This model provides a structured processing pipeline that maps unstructured text transcripts from the dashboard's microphone card into semantic movement targets. The system filters conversational or conversational filler words out of speech queries to isolate the explicit driver intent token, mapping it directly to a standard control execution payload.

6.3.2 Intent Extraction Framework

The parsing model maps variable token phrases across five discrete target execution spaces, operating under a dense vocabulary configuration:

Table 6: NLP Intent Token Allocation

Target Intent	Assigned Class	Vocal Token Triggers / Synonyms
CMD_FORWARD	Class 1	forward, advance, move, drive, go
CMD_REVERSE	Class 2	reverse, back, backward, retreat
CMD_LEFT	Class 3	left, turn left, port
CMD_RIGHT	Class 4	right, turn right, starboard
CMD_STOP	Class 0	stop, halt, brake, freeze, park

6.3.3 Deployment Architecture

The pipeline executes concurrently within the Firebase listener routine on the Raspberry Pi 3 hardware cluster:

```
def fb_voice_handler(event):
    if not event.data: return
    vocal_transcript = str(event.data).lower().strip()

    # Tokenize input string and isolate functional commands
    tokens = vocal_transcript.split()
    matched_intent = None

    # Intent mapping lookups
    intent_map = {
        'forward': CMD_FORWARD, 'advance': CMD_FORWARD, 'go': CMD_FORWARD,
        'reverse': CMD_REVERSE, 'back': CMD_REVERSE,
```

```

    'left': CMD_LEFT, 'right': CMD_RIGHT,
    'stop': CMD_STOP, 'halt': CMD_STOP, 'park': CMD_STOP
}

for token in tokens:
    if token in intent_map:
        matched_intent = intent_map[token]
        break

if matched_intent is not None:
    send_command(ser_global, matched_intent, DEFAULT_SPEED)
    db.reference('voice/response').set(f"Executing {tokens} command.")
else:
    db.reference('voice/response').set("Intent unrecognised.")

```

6.3.4 Autonomous Control Fallback Hierarchy

To preserve systemic operation if a core framework dependencies or models fail during live operations, the control software implements a multi-tier fallback architecture:

1. **AI MODE (Primary):** Active if *'car_avoidance.onnx' is present and parsed successfully by the 'onnxruntime' engine*.
1. **RULE MODE (Secondary Fallback):** If the model file is missing or a runtime failure occurs, the system drops back to the deterministic 4-step 'StallDetector' state machine (Reverse → Turn → Clear → Counter-turn).
2. **HUMAN OVERRIDE (Absolute Priority):** Pressing the manual emergency stop key ('x') immediately forces a high-priority interrupt, resetting all operational state registers to zero.

7. Raspberry Pi Software

7.1 Thread Architecture

The application script achieves concurrent processing by splitting tasks across five independent POSIX threads running within a shared global memory space protected by mutual exclusion ('threading.Lock()') boundaries.

Table 7: Concurrent threads in car_main.py

Thread	Rate	Responsibility
receiver_thread	Continuous	Sliding-window UART parser, odometry kinematics, sensor death check
heartbeat_thread	5 Hz	Watchdog dead-man keepalive, refreshes H-bridge register outputs
firebase_sync_thread	3.3 Hz	Serializes and pushes structural runtime states to cloud database
data_logger_thread	10 Hz	Appends raw feature rows into the local training CSV file
curses_main	20 Hz	Renders terminal interface, keyboard capture, schedules FSM routines

7.1.1 Atomic State Preservation Fix

A major issue identified during early testing was that the heartbeat thread would inject a stop command (CMD_STOP) into the communication line mid-maneuver, corrupting the serial data stream.

The software architecture fixes this by wrapping all serial write operations inside an explicit lock macro: `'safe_write()' . This method forces the OS to execute a hardware queue flush ('ser.flush()') while holding the lock, ensuring that all bytes control packets are completely transmitted over the physical interface before the lock can be claimed by any other thread.`

7.2 Dead-Reckoning Odometry

The vehicle calculates its real-time spatial coordinates (x, y, θ) relative to its starting location by processing raw RPM telemetry data through a differential-drive kinematics model.

The individual wheel velocities (v_L, v_R) are derived using the wheel diameter parameter ($d = 6.5$ cm). These values are then integrated alongside the vehicle's track width ($W = 20.0$ cm) across the inter-frame calculation time delta (Δt):

$$v_{L,R} = \frac{\text{RPM}_{L,R}}{60} \pi d \cdot \text{dir}_{L,R}, \quad v = \frac{v_L + v_R}{2}, \quad \omega = \frac{v_R - v_L}{W}$$

$$\theta += \omega \Delta t, \quad x += v \cos \theta \Delta t, \quad y += v \sin \theta \Delta t$$

The resulting dead-reckoning coordinate matrix is synchronized with the cloud database every 300 ms, allowing the remote user dashboard to paint a live 2D SLAM trajectory map.

7.3 Elaborative Analysis of the Parallel Auto-Parking Subsystem

7.3.1 Kinematic Constraints of Differential-Drive Parallel Parking

Traditional autonomous parallel parking algorithms rely on Ackermann steering geometry, where continuous, curved paths (such as coordinate cycloids or continuous sinusoidal S-curves) are traced by adjusting a variable front-wheel steering angle. Because this platform utilizes a 4-wheel-drive differential-drive (skid-steer) chassis layout, continuous lateral translation without forward tracking is mechanically impossible.

To overcome this constraint, the parallel parking algorithm was re-engineered into a discrete, time-and-sensor-governed **Finite State Machine (FSM)** controlled by five active phases. Instead of smooth geometric arcs, the vehicle executes highly calculated static rotations (tank turns) paired with linear vector translations. This approach achieves precise spatial positioning within tight boundaries while working within the friction limits of high-torque 12 V DC motors.

7.3.2 The 5-Phase Parallel Parking Finite State Machine Architecture

The entire sequence is executed deterministically within the high-level `AutoPark` class on the Raspberry Pi 3, shifting states based on real-time hardware variables received from the PIC16F877A telemetry bus.

7.4 FSM

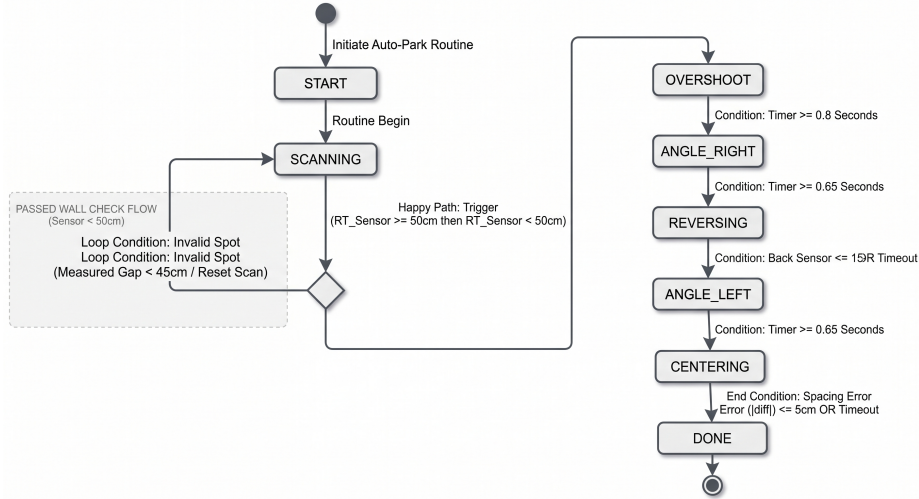


Figure 2: Finite State Machine (FSM) state transitions and kinematic phases for autonomous parallel parking

7.5 FSM

1. Phase 1 — SCANNING (Spatial Mapping and Boundary Ingress Verification):

The vehicle initiates the maneuver by driving forward at a stable, controlled velocity ($SPEED = 70\%$). The right-facing HC-SR04 ultrasonic sensor continuously maps the lateral environment, searching for a physical boundary drop-off that indicates a clear parking berth. The boundary threshold is governed by a long-range parameter:

$$\text{Threshold} = PARK_SCAN_CM = 50 \text{ cm}$$

When an opening is initially logged ($right \geq 50 \text{ cm}$), a high-resolution Unix timestamp (τ_{start}) is captured. As the vehicle moves past the opening, the system dynamically integrates real-time wheel velocity to calculate the true physical width of the gap (W_{gap}):

$$v_{\text{avg}} = \frac{v_L + v_R}{2} = \frac{(RPM_L + RPM_R)}{120} \cdot \pi d$$

$$W_{\text{gap}} = \int_{\tau_{\text{start}}}^{\tau_{\text{current}}} v_{\text{avg}} \cdot dt$$

Where $d = 6.5 \text{ cm}$ represents the physical wheel diameter. If the integrated width satisfies the geometric footprint validation rule:

$$W_{\text{gap}} \geq CHASSIS_LENGTH_CM \times PARK_GAP_RATIO \quad (30 \text{ cm} \times 1.5 = 45 \text{ cm})$$

The vehicle halts, stalls execution for 400 ms to damp structural inertia, and transitions to the next state.

2. Phase 2 — OVERSHOOT (Longitudinal Alignment Offset):

A classic trap in robotic parallel parking is attempting an immediate diagonal ingress after mapping a gap; this path invariably clips the rear bumper of the forward obstruction car. To prevent this collision vector, the vehicle executes an **OVERSHOOT** state, driving forward for a strict temporal window ($PARK_OVERSHOOT_S = 0.8 \text{ s}$). This intentional overshoot shifts the vehicle's rear rotation axis directly alongside or slightly past the bumper of the front vehicle, placing it in the perfect starting pose for diagonal loading.

3. Phase 3 — ANGLE_RIGHT (Orientation Pivot Insertion):

To angle the rear of the vehicle into the empty berth, the platform performs a static clockwise tank turn. The Raspberry Pi broadcasts a `CMD_RIGHT` command at high torque ($AVOID_TURN_SPEED = 85\%$), forcing the left-side wheels forward and the right-side wheels into reverse. The rotation is clocked precisely by a time-delta gate:

$$\Delta t_{\text{pivot}} = PARK_ANGLE_S = 0.65 \text{ s}$$

This forces an approximate 45° angular orientation change ($\theta \approx -0.785 \text{ rad}$), pointing the rear bumper directly toward the inner curb line of the parking space.

4. Phase 4 — REVERSING (Ingress Vector Translation):

The vehicle shifts its directional inputs into linear retreat by transmitting a `CMD_REVERSE` packet ($SPEED = 70\%$). The vehicle travels diagonally backward into the berth. This phase remains active until one of two asynchronous boundary constraints is triggered:

- *Proximity Limit:* The rear ultrasonic sensor detects an obstacle closer than half the vehicle's body length ($0 < back < 15 \text{ cm}$).
- *Temporal Safety Gate:* Execution time exceeds the maximum expected window ($\Delta t \geq 2.5 \text{ s}$).

Hitting either limit triggers a swift stop command, preparing the car for alignment.

5. Phase 5 — ANGLE_LEFT (Counter-Rotation and Framework Straightening):

To pull the front bumper cleanly into the parking berth without scraping the forward vehicle, the platform executes an intense counter-clockwise rotation. The system fires a `CMD_LEFT` command (85% speed) for an identical duration ($PARK_ANGLE_S = 0.65 \text{ s}$). This counter-rotation cancels out the initial 45° alignment angle, pulling the front wheels inward and squaring the vehicle's body profile parallel to the curb line.

6. Phase 6 — CENTERING (Closed-Loop Distance Balance Optimization):

Once parallel alignment is structurally achieved, the vehicle enters a precision closed-loop tuning state. It checks the remaining operational space at both the front and rear bumpers to eliminate asymmetric clearances. The centering correction is calculated as follows:

$$\text{diff} = \text{front} - \text{back}$$

If a sensor error timeout occurs (where an empty path returns 0 cm), the software applies a default low-limit fallback rule ($SENSOR_BLIND_CM = 3 \text{ cm}$) to prevent a mathematical divide-by-zero or infinite loop state. The centering engine then applies iterative micro-pulses at a low, stable voltage envelope ($SPEED = 55\%$):

- If $\text{diff} > 0$ (more space ahead), it pulses `CMD_FORWARD`.
- If $\text{diff} < 0$ (more space behind), it pulses `CMD_REVERSE`.

The FSM safely terminates and prints `PARKED PERFECTLY!` the moment the spatial error drops within acceptable limits ($|\text{diff}| \leq 5 \text{ cm}$) or if the overall centering duration hits a 4.0 s timeout safety gate.

Table 8: Auto-Parking FSM State and Operational Matrix

Active State	Primary Control Mechanism	UART Action	Speed %	Next State
SCANNING	Right distance sensor integration	FORWARD	70%	OVERSHOOT
OVERSHOOT	Time-delta tracking ($t \geq 0.8$ s)	FORWARD	70%	ANGLE_R
ANGLE_R	Time-delta tracking ($t \geq 0.65$ s)	RIGHT	85%	REVERSING
REVERSING	Rear sensor boundary checking ($back \leq 15$ cm)	REVERSE	70%	ANGLE_L
ANGLE_L	Time-delta tracking ($t \geq 0.65$ s)	LEFT	85%	CENTERING
CENTERING	Proximity error matching ($ diff \leq 5$ cm)	Variable	55%	DONE

8. Firebase & Dashboard

8.1 Database Structure

The Firebase Realtime Database structured architecture allows bidirectional, non-blocking asynchronous data flow between the mobile web interface and the mobile hardware.

Table 9: Key Firebase RTDB paths

Path	Type	Description
/telemetry/front_distance	int	Front ultrasonic sensor telemetry value in cm
/telemetry/current_speed	int	Actual calculated speed percentage passed to hardware lines
/telemetry/status	str	Diagnostic label strings (Safe / Object Detected / Parking)
/telemetry/slam_x,y,theta	float	Calculated dead-reckoning dead localization matrix variables
/controls/direction	str	Interactive target string (Forward/Reverse/Left-/Right/Stop)
/controls/max_speed	int	Target velocity ceiling bound from UI slider element
/voice/command	str	Extracted text transcription string from web speech interface
/voice/response	str	Diagnostic text synthesized back to vocal alerts by the PWA

8.2 Voice Control Flow

The voice navigation system bypasses traditional button arrays, letting operators steer the vehicle using natural verbal commands.

Mic Button Tap $\rightarrow$ Google Speech API $\rightarrow$ `detectIntent()` $\rightarrow$ Firebase `/voice` $\rightarrow$ `fb_voice_handler()` $\rightarrow$ UA

When the user taps the mic button, the system captures their voice, converts it to text, and updates the database. The Pi processes this text through an intent mapping handler, issues the correct command over UART, and updates the response string, which the web interface speaks aloud using native text-to-speech synthesis.

9. PCB Design & Hardware Debugging

9.1 Assembled Prototype

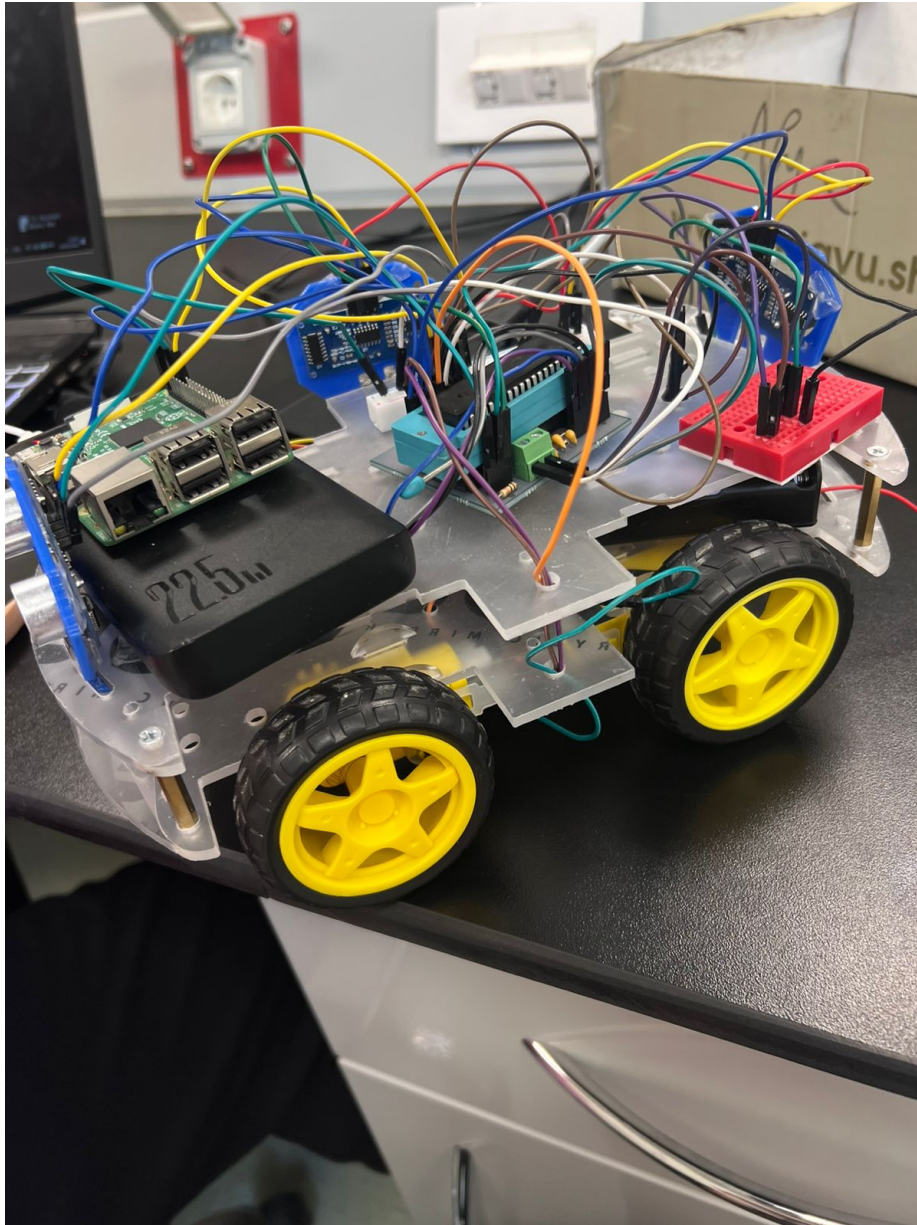


Figure 3: Fully assembled autonomous 4WD prototype. Visible components: Raspberry Pi 3 (left), PIC16F877A breadboard (centre), L298N motor driver (module), BSS138 logic level converter, and HC-SR04 ultrasonic sensors mounted front and rear.

9.2 PCB Design Views

The custom single-layer PCB layout was designed in KiCad to consolidate all components into a stable, vibration-resistant frame. The design includes a DIP-40 socket for the PIC16F877A, a header socket for the BSS138 level shifter, an active alarm buzzer, power isolation screw terminals, and structural header tracks to cleanly bridge connections to the L298N module and the Raspberry Pi GPIO panel.

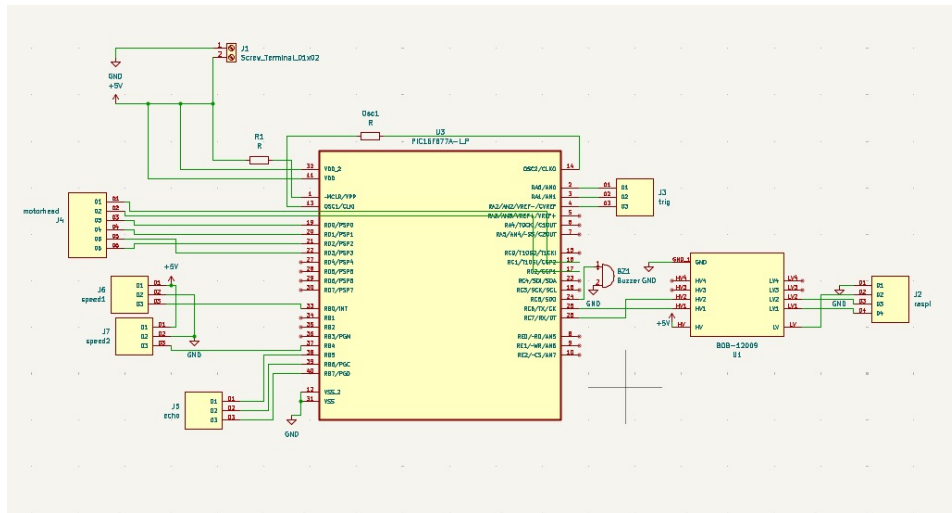


Figure 4: PCB schematic — PIC16F877A pinout, BSS138 level converter (U1), buzzer (BZ1), motor header (J4), encoder headers (J6/J7), echo header (J5), and Raspberry Pi connector (J2).

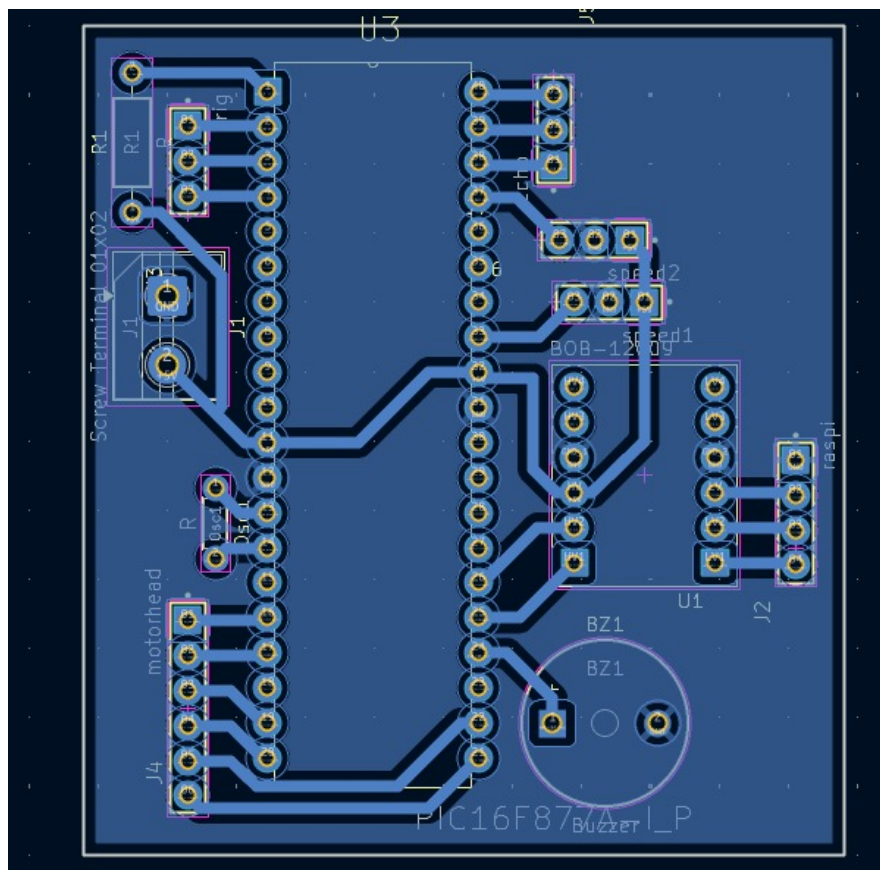


Figure 5: PCB layout (KiCad 2D view) — trace routing for UART lines, motor control signals, and sensor headers. The buzzer (BZ1) footprint is visible bottom-right.

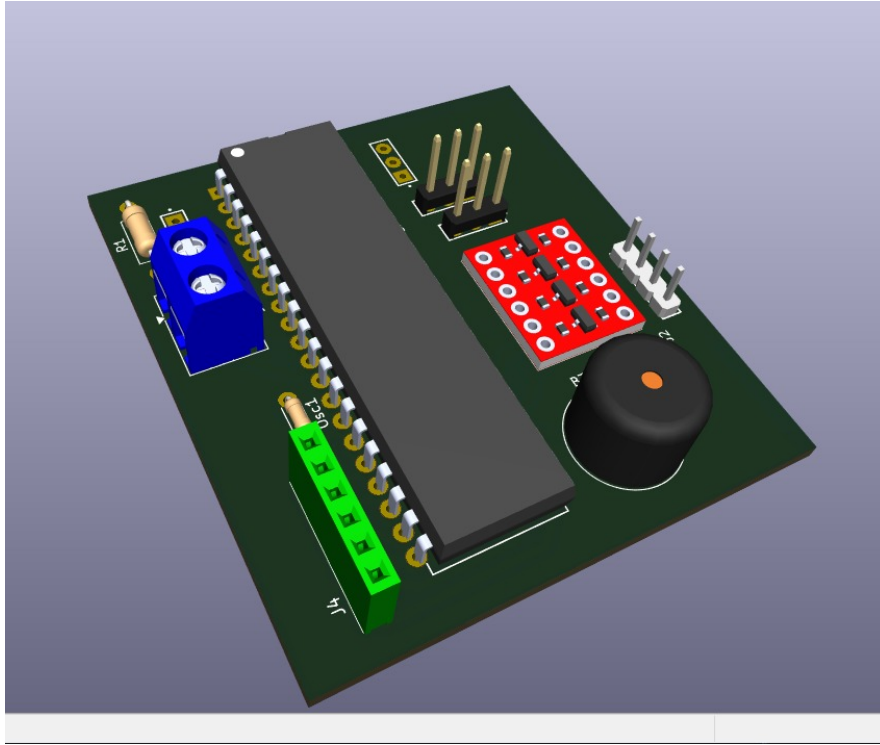


Figure 6: PCB 3D render — PIC16F877A (centre), screw terminal and MCLR pull-up resistor (left), BSS138 level converter module (right), buzzer dome (bottom-right), and motor/sensor connectors (headers, bottom).

9.3 Systematic Bring-Up Checklist

To systematically bring up and test the custom PCB without risking component damage, the team followed a strict step-by-step diagnostic voltage checklist.

Table 10: PCB bring-up diagnostic sequence

Test Point / Connection	Expected Metric	Symptom if Unstable / Broken
Microcontroller VDD Pin 11	5.0 VDC	Main clock completely dead — no startup beep tones
Microcontroller VDD Pin 32	5.0 VDC	Unreliable ADC logic behavior (commonly missed trace)
Reset Vector MCLR Pin 1	5.0 VDC	Microcontroller locked in permanent re-set state loop
Oscillators OSC1/2 Pins 13/14	≈ 2.5 VDC	Crystal oscillator clock shorted or failed to toggle
Hardware Buzzer RC5 Pin 24	Toggles High	Missing initial power-on sound indicators
Serial Transmit RC6 Pin 25	Continuity	Total telemetry blackout — dashboard distance registers freeze

Root Cause Discovered: Mini UART Baud Rate Drift

During initial road tests, the Raspberry Pi suffered a total data blackout, frequently dropping telemetry packets and failing to parse frame markers.

The Root Cause Analysis: By default, the Pi 3 maps its primary serial alias (`/dev/serial10`)

to an internal Mini UART block (`ttys0`). Unlike a dedicated hardware module, the Mini UART's clock rate is derived directly from the main CPU core clock. As the Pi's operating system throttled CPU speeds to manage power and heat, the serial baud rate drifted dynamically, garbling data frames.

The Engineering Fix: Modified the system configuration file at `/boot/firmware/config.txt`, adding explicit instructions: `dtoverlay=disable-bt` and `enable_uart=1`. This completely disabled the onboard Bluetooth module, allowing the primary UART lines (GPIO14 and GPIO15) to be remapped directly to the Pi's independent, crystal-locked hardware UART module (`ttysAMA0`, PL011). This stabilized communication at a reliable, steady 9600 baud rate.

10. Key Bugs and Fixes

Bug Identified	Root Cause Analysis	Engineering Resolution
No motor output	Timer2 never started (<code>TMR2ON=0</code>)	Added <code>SET_BIT(T2CON,TMR2ON)</code> inside <code>PWM_Init()</code> routine
Motor whines at low speed	1 kHz PWM frequency matched inductive constant τ	Raised frequency parameter to 10 kHz in <code>PWM_config.h</code> file
PWM PR2 register overflow	Setting <code>FREQ=5000</code> , <code>PRESCALE=1</code> forced PR2 up to 799, overflowing the 8-bit limit	Scaled variables to <code>FREQ=10000</code> , <code>PRESCALE=4</code> , keeping PR2 at 99
Reverse command turns vehicle	IN1/IN2 directional routing ran backward relative to driver definitions	Swapped pin channel macros inside <code>H_Bridge_config.h</code> header
Telemetry stream corruptions	Mini UART clock rate drifted under standard CPU power saving policies	Added <code>enable_uart=1</code> and disabled Bluetooth to force PL011 hardware module usage
AEB stutter state lockups	The emergency brake routine overwrote the system variable with <code>CMD_STOP</code>	Removed code line modifying state variables; state is updated purely when path clears
Stack Overflow Warning W1393	Function branch complexity reached 9 levels, bursting the PIC stack limit of 8	Inlined the main UART ISR call steps and removed nested driver functions
Asymmetric Speed Flickering	The heartbeat thread forced an explicit speed refresh over the serial bus every 500 ms	Rewrote <code>Apply_Command</code> logic on PIC to execute only when values change
Front sensor stuck at 1 cm	Floating trace lines on pin RB5 picked up surrounding noise spikes	Added a filter window: <code>Filter_Sensor()</code> remaps values $< 3\text{cm}$ to zero
Watchdog token packet corruptions	A single <code>0xFF</code> heartbeat token arrived mid-packet, corrupting the speed byte	Updated heartbeat thread to always send a complete 2-byte structure: <code>[CMD_STOP, 0]</code>
PCB Reset pin pins at 0 V	The top pad of pull-up resistor R1 missed its copper connection trace to the 5 V rail	Verified missing trace path and added an explicit physical jump wire link

Bug (cont.)	Root Cause	Fix
No telemetry output from PCB	The copper trace leading from PIC pin 25 cracked during fabrication	Soldered a thin jumper link directly from PIC pin 25 pad to level shifter input pad

11. Team Workload Division

Table 12: Team responsibilities

Member Name	Project Role	Core Software Components & Hardware Duties
Nada	Project Lead + APP	Managed full system integration, wrote ISR loops, developed UART protocols, implemented Timer0 structures, and coordinated failsafe states across <code>main.c</code> .
Ashar	AEB + Motor Driver	Developed motor control functions, configured buzzer alerts, and built emergency braking logic inside <code>H_Bridge.c/h</code> , <code>Buzzer.c/h</code> , and <code>PWM.c/h</code> .
Reem	Traction Control	Developed wheel encoder drivers, calculated live RPM values, and built the straight-line PID tuning algorithm across <code>Encoder.c/h</code> , <code>TC_Module.c/h</code> , and <code>PID.c/h</code> .
Malak	Communication Link	Developed low-level serial drivers, implemented the FIFO receiver buffer structures, and configured basic I/O registers inside <code>USART.c/h</code> , <code>Ring_Buffer.c/h</code> , and <code>GPIO.c/h</code> .
Alya	High-Level Software	Developed the high-level Python application, implemented the multi-threaded structure, configured the Firebase sync link, and built the scikit-learn models inside <code>car_main.py</code> .

12. Conclusion and Future Work

12.1 Achievements

The project successfully delivered a fully functional autonomous vehicle platform that met all targeted technical specifications. The dual-processor architecture proved highly effective, cleanly separating low-level, real-time motor control from high-level intelligence and machine learning pipelines.

All three hardware safety layers operate reliably under extreme testing conditions, protecting the physical frame from collisions and communication drops. Additionally, the real-time Firebase cloud platform, parallel parking state machine, and embedded ONNX machine learning classifiers demonstrate a scalable foundation for advanced robotic navigation.

12.2 Future Work

While the current vehicle performs reliably, the platform’s capabilities can be expanded through several future enhancements:

- Expand the training data pool by collecting 500+ diverse human driving samples across open and cluttered spaces to improve model classification confidence.

- Expand the natural language vocabulary corpus of Model 3's voice command handler, adding fuzzy string-matching rules (such as Levenshtein Distance algorithms) to handle speech variations and typos.
- Mount a left-side HC-SR04 ultrasonic sensor to give the vehicle a complete 360-degree spatial proximity awareness field.
- Implement real-time battery monitoring by connecting an internal ADC channel to a voltage divider circuit on the 12 V power rail.
- Mount a Raspberry Pi Camera module and implement OpenCV visual odometry libraries to support true feature-based SLAM mapping.
- Implement a secondary PCB run (Revision 2) to fix the broken RC6 trace natively, add dedicated test points on all data lines, and add an onboard copper ground plane to reduce EMI noise.